

Dormancy Engineering

Software Design for the Long Sleep

P-RICK RESEARCH PROGRAM

an independent working-paper series on missing software

working paper P-019 v1.0 • series V

October 7, 2026 • cc by 4.0

Abstract — Software engineering has two mature regimes for the passage of time, and the space between them is empty. The always-on regime assumes continuous exercise: continuous integration, canaries, synthetic checks, chaos drills — systems prove themselves by never stopping. The archival regime assumes permanent retirement: Software Heritage and its peers preserve sources for historians, treating executability as a bonus. Between them lies the regime this paper names and specifies: **dormancy** — software that must *work* after years of not running. Password-recovery kits and break-glass runbooks, appliance diagnostic firmware, amateur-radio emergency stacks, air-gapped recovery environments, cold storage wallets, legal-hold evidence systems, escrowed update packages, the earthquake kit’s USB boot stick: artifacts exercised exactly when everything else has already failed, and therefore guaranteed to be exercised *unexercised*. The failure physics of dormancy is enumerable: dependency capability drift (the TLS cipher, the OS API, the cloud endpoint that left), credential expiry (certificates, tokens, and signing keys that die on the clock while nobody looks), media decay, and knowledge decay (the runbook’s readers have left). Yet no discipline measures wake-time assurance, declares dormancy classes, or schedules the escalating probes that keep the promise “it will be there when you need it” falsifiable. This paper specifies that discipline: dormancy classes D1–D3 with declared wake-latency bounds; the environment-drift ledger enumerating capability dependencies and their observed removal rates; wake probes at three depths; the exercise calendar; the readiness contract with its insurance-grade audit form; and the economics that price dormant-but-warranted against the naive alternative of leaving everything running. Every component exists in adjacent worlds — SRE continuous verification, OAIS archival science, MIL-grade storage standards, the seed bank’s germination testing, the military’s monthly-check tradition — and no system composes them into a general engineering discipline for sleeping software. The claim is falsifiable by a single system that declares a dormancy class, schedules its own probes, and wakes within its promised bound after five cold years. **Keywords:** dormancy, cold standby, wake-time assurance, bit-rot, capability drift, wake probes, exercise calendar, readiness contract, emergency software

Keywords: dormancy, cold standby, wake-time assurance, bit-rot, capability drift, wake probes, exercise calendar, readiness contract, emergency software

p-rick working paper P-019 · series V (promises) · draft 1.0

Software engineering has two mature regimes for the passage of time, and the space between them is empty. The always-on regime assumes continuous exercise: continuous integration, canaries, synthetic checks, chaos drills — systems prove themselves by never stopping. The archival regime assumes permanent retirement: Software Heritage and its peers preserve sources for historians, treating executability as a bonus. Between them lies the regime this paper names and specifies: **dormancy** — software that must *work* after years of not running. Password-recovery kits and break-glass runbooks, appliance diagnostic firmware, amateur-radio emergency stacks, air-gapped recovery environments, cold storage wallets, legal-hold evidence systems, escrowed update packages, the earthquake kit’s USB boot stick: artifacts exercised exactly when everything else has already failed, and therefore guaranteed to be exercised *unexercised*. The failure physics of dormancy is enumerable: dependency capability drift (the TLS cipher, the OS API, the cloud endpoint that left), credential expiry (certificates, tokens, and signing keys that die

Dormancy Engineering

Software Design for the Long Sleep

P-RICK RESEARCH PROGRAM

an independent working-paper series on missing software

working paper P-019 · v1.0 · series V

October 9, 2026 · cc by 4.0

Abstract — Software engineering has two mature regimes for the passage of time, and the space between them is empty. The always-on regime assumes continuous exercise: continuous integration, canaries, synthetic checks, chaos drills — systems prove themselves by never stopping. The archival regime assumes permanent retirement: Software Heritage and its peers preserve sources for historians, treating executability as a bonus. Between them lies the regime this paper names and specifies: **dormancy** — software that must *work* after years of not running. Password-recovery kits and break-glass runbooks, appliance diagnostic firmware, amateur-radio emergency stacks, air-gapped recovery environments, cold storage wallets, legal-hold evidence systems, escrowed update packages, the earthquake kit’s USB boot stick: artifacts exercised exactly when everything else has already failed, and therefore guaranteed to be exercised *unexercised*. The failure physics of dormancy is enumerable: dependency capability drift (the TLS cipher, the OS API, the cloud endpoint that left), credential expiry (certificates, tokens, and signing keys that die on the clock while nobody looks), media decay, and knowledge decay (the runbook’s readers have left). Yet no discipline measures wake-time assurance, declares dormancy classes, or schedules the escalating probes that keep the promise “it will be there when you need it” falsifiable. This paper specifies that discipline: dormancy classes D1–D3 with declared wake-latency bounds; the environment-drift ledger enumerating capability dependencies and their observed removal rates; wake probes at three depths; the exercise calendar; the readiness contract with its insurance-grade audit form; and the economics that price dormant-but-warranted against the naive alternative of leaving everything running. Every component exists in adjacent worlds — SRE continuous verification, OAIS archival science, MIL-grade storage standards, the seed bank’s germination testing, the military’s monthly-check tradition — and no system composes them into a general engineering discipline for sleeping software. The claim is falsifiable by a single system that declares a dormancy class, schedules its own probes, and wakes within its promised bound after five cold years. **Keywords:** dormancy, cold standby, wake-time assurance, bit-rot, capability drift, wake probes, exercise calendar, readiness contract, emergency software

Keywords: dormancy, cold standby, wake-time assurance, bit-rot, capability drift, wake probes, exercise calendar, readiness contract, emergency software

Software engineering has two mature regimes for the passage of time, and the space between them is empty. The always-on regime assumes continuous exercise: continuous integration, canaries, synthetic checks, chaos drills — systems prove themselves by never stopping. The archival regime assumes permanent retirement: Software Heritage and its peers preserve sources for historians, treating executability as a bonus. Between them lies the regime this paper names and specifies: **dormancy** — software that must *work* after years of not running. Password-recovery kits and break-glass runbooks, appliance diagnostic firmware, amateur-radio emergency stacks, air-gapped recovery environments, cold storage wallets, legal-hold evidence systems, escrowed update packages, the earthquake kit’s USB boot stick: artifacts exercised exactly when everything else has already failed, and therefore guaranteed to be exercised *unexercised*. The failure physics of dormancy is enumerable: dependency capability drift (the TLS cipher, the OS API, the cloud endpoint that left), credential expiry (certificates, tokens, and signing keys that die on the clock while nobody looks), media decay, and knowledge decay (the runbook’s readers have left). Yet no discipline measures wake-time assurance, declares dormancy classes, or schedules the escalating probes that keep the promise “it will be there when you need it” falsifiable. This paper specifies that discipline: dormancy classes D1–D3 with declared wake-latency bounds; the environment-drift ledger

enumerating capability dependencies and their observed removal rates; wake probes at three depths; the exercise calendar; the readiness contract with its insurance-grade audit form; and the economics that price dormant-but-warranted against the naive alternative of leaving everything running. Every component exists in adjacent worlds — SRE continuous verification, OASIS archival science, MIL-grade storage standards, the seed bank’s germination testing, the military’s monthly-check tradition — and no system composes them into a general engineering discipline for sleeping software. The claim is falsifiable by a single system that declares a dormancy class, schedules its own probes, and wakes within its promised bound after five cold years.

1 Introduction

There is a boot stick in a drawer in a data center in Mumbai, assembled the week the world learned what ransomware does to hospitals. It contains a minimal OS, an image of the department’s systems, and a laminated card with six steps. It has not been booted since. Somewhere in the same city, an amateur radio club maintains a packet-radio firmware image for the regional emergency network — refreshed annually in principle, untouched in practice since the drill of three years ago, because drills are volunteer weekends and winters are short. In a law firm’s evidence vault, a matter-closed archive holds encrypted containers and the e-discovery tool that reads them, both seized under legal hold and both now four years into a sleep that may end in a courtroom. And on a continent with more seismic optimism than budget, a school keeps a USB stick of offline enrollment software in a steel box marked *earthquake kit*.

These artifacts share one property more important than any of their differences: they are *guaranteed to be exercised exactly when everything else has failed*. The boot stick boots when the hospital is down; the radio firmware configures when the network is gone; the evidence tool decrypts when a judge orders production; the enrollment software enrolls when the buildings are unsafe. Their moment of maximum dependence is precisely their moment of minimum preparation — and the years between acquisition and use are not neutral. They are an active adversary: certificates expire on schedule, TLS libraries forget the ciphers that old endpoints still speak, operating systems remove the syscall that the recovery tool links, cloud endpoints move or die, media accumulates read errors, and the six laminated steps slowly outlive the people who could have debugged step four. The drawer is not a vault. It is a slow-fire furnace with excellent marketing.

The industry’s answer to the passage of time has hardened into two regimes. The **always-on regime** keeps software alive by exercising it: CI runs the tests nightly, synthetic checks hit the endpoints hourly, chaos engineering kills components deliberately so that failure is rehearsed before it is real. The approach works — for systems whose owners can afford to run them forever. The **archival regime** accepts that software stops and preserves its *sources*: Software Heritage harvests repositories; emulation stacks (emulators, VM images, container snapshots) promise that *somewhere*, on *some* maintained hypervisor, the artifact could run again. The approach works — for historians, whose deadlines are generous.

Neither regime serves the drawer. The boot stick cannot run continuously — its entire value is being offline, unreachable by the adversary that motivates it; and it is not a source tree to be recompiled on demand — its entire value is turning on with *zero* build infrastructure, in a building that may not have network, let alone a toolchain. Dormant software must be *both* preserved (like an archive) and *operational* (like a service), while being *neither* maintained daily (like CI) nor freely re-derivable (like a repo). That intersection — operational, preserved, unmaintained, and *warranted* — is an engineering regime with no name, no metrics, and no discipline. This paper gives it all three.

The series context: series V — *promises software implies and never has to honor*. The promise here is the quietest one software ever makes: *when you need me, I will turn on*. No interface states it, no SLO covers it, and nothing anywhere measures whether it is true. Section 2 defines dormancy and its failure physics. Section 3 surveys the adjacent worlds and verifies the gap. Section 4 analyzes why the empty regime stayed empty. Section 5 specifies the discipline. Sections 6–10 close with evaluation, boundary, objections, limitations, and conclusions.

2 The physics of sleeping software

2.1 Dormancy classes

Define dormancy operationally: a system is **dormant** at time t if it is *not running* and its *next intended use* is conditional and future. The always-on/archival regimes bookend the spectrum; the interesting middle is stratified by the *maximum wake latency* the owner can tolerate:

- **D1 — warm standby** (wake bound: minutes to hours). Cold spares in a cluster, secondary regions, failover appliances. Verification cadence: continuous-to-weekly (this class borders the always-on regime, and SRE practice genuinely covers parts of it — it is included for completeness of the ladder, not as the gap).
- **D2 — cold standby** (wake bound: hours to days, usable quarterly). The break-glass runbook, the cold-site rebuild kit, the offline enrollment stack, the sealed diagnostic firmware. Exercise cadence: quarterly-to-annual. *This is the gap's center of mass.*
- **D3 — deep dormancy** (wake bound: days, usable on demand). Legal-hold evidence systems, es-crowed updates under multi-year release terms, cold wallets with paper procedures, seed-vault-grade software artifacts. Exercise cadence: annual-to-multi-year, with *probe* (non-invasive) substitutions between full exercises.

The classes are not flavors but *contracts*: a class declares who is allowed to forget what, how fast, and at what verification price. Class D2 is where the economics bite — frequent enough that rot accumulates, rare enough that nobody remembers the details.

2.2 The four decay channels

What actually kills dormant software is enumerable, and the enumeration matters because each channel has a distinct sensor and a distinct countermeasure:

1. **Capability drift** — the environment removes what the artifact depends on. The TLS handshake of 2018 is refused by the endpoint of 2026; the syscall is gone from the kernel; the cloud API's v2 endpoint returned 404 sometime in 2023; the firmware image assumes a bootloader partition layout that the vendor silently changed. This is the channel with the *largest measured surface* in the wild, and the one cloud-native architecture accelerated: the assumption of continuous reachability made endpoint churn someone else's cost — until the dormant artifact wakes and finds the address vacant.
2. **Credential expiry** — secrets die on the clock. Code-signing certificates, TLS chains, API tokens, and encryption subkeys all carry lifetimes chosen for *continuously-maintained* systems, where rotation is routine. A dormant artifact inherits short-lived credentials with no rotation loop: the wake proceeds, the handshake fails, and the failure is discoverable only by waking. The 2021 mass breakage from a widely-trusted root certificate's expiry — old devices dropping off the web they still nominally spoke — is the public demonstration that clock-time kills stopped systems (event details: verification queued; the class of event is the point, not the instance).
3. **Media decay** — the physical layer rots. Flash loses charge over years; the celebrated archival USB stick is among the *worst* storage media for dormancy, a fact almost perfectly inverse to its popularity in emergency kits; optical media outlasts it by an order of magnitude; paper's century-plus half-life embarrasses every digital competitor. Multi-copy, multi-media storage with checksums is the partial answer — but a checksum proves the bits; it says nothing about whether the bits still *run* (the distinction Section 3's archival science knows well).
4. **Knowledge decay** — the humans leave. The runbook's author changes jobs; the six laminated steps reference an internal wiki that was migrated twice; the only engineer who knew why step four has a 90-second timeout is now a manager in another time zone. The bus-factor protocol (P-010) formalized

this for package maintainers; here it recurs at the scale of a single laminated card, where the bus factor is usually one from day one.

2.3 Wake-time assurance

The measurable object: **wake-time assurance** $W(D, T) = P(\text{system meets its declared service contract within its declared wake bound, after } T \text{ time dormant})$. The definition does the same work SRE's error budgets did for availability: it converts an aspiration into a number a contract can hold. Declaring a dormancy class is committing to a W floor; failing a scheduled exercise is burning the budget; and the readiness contract of Section 5.5 prices the difference. Note what the definition implies: W is *conditional on the environment's* churn — a D2 artifact with identical bits can have $W = 0.98$ in one infrastructure and $W = 0.4$ in another, because capability drift is a property of the world, not the artifact. Dormancy engineering is therefore intrinsically an *actuarial* discipline: it models the environment's removal rates (Section 5.2) the way reliability engineering models component failure rates — the artifact is a component, the world is the machine.

3 The landscape: mature on both sides, empty in the middle

SRE and continuous verification. The always-on regime's discipline: SLIs and SLOs, canaries, synthetic monitoring, game days, chaos engineering. Its methods measure and maintain *running* systems; its tools assume telemetry exists. A dormant artifact has no telemetry by definition — it emits nothing, so it cannot page, so the monitoring tradition has no object to attach to. The discipline's one transferable gift is the exercise philosophy: rehearse failure deliberately. Verdict: the philosophy transfers; the machinery does not apply.

Software preservation: Software Heritage, emulators, OAIS. The archival regime's discipline, and it is genuinely rigorous — OAIS (ISO 14721) specifies ingest, storage, and *designated communities*; Software Heritage's content-addressed vault preserves sources at scale; emulation research (Emularity, EaaS) demonstrates re-execution of decades-old artifacts. The boundary is in the *guarantee*: archival guarantees *preservation*, with executability as a research goal, and its success criterion is scholarly access under no time pressure. A courtroom's 48-hour production order, a hospital's down week, or an emergency network's 4-hour activation target are not in the model. Verdict: the preservation half of dormancy, mature and adoptable; the operational half, absent.

MIL storage standards and shelf-life engineering. The tradition of equipment that must work after decades in a crate: storage-condition specs, preservation packaging, periodic inspection cycles, the military's culture of monthly checks. This is the closest *institutional* precedent — maintenance-by-calendar rather than maintenance-by-telemetry — and the exercise calendar of Section 5.4 is its direct transplant into software terms. Verdict: the precedent proves institutions can run calendar-based assurance programs for decades; the specifics never crossed to software artifacts.

Seed banks and germination testing. Svalbard's operators periodically germination-test stored seed lots to measure *viability*, not just presence: the analogue is exact — the archive's integrity check (checksum) versus the archive's *viability check* (does it grow). The paper's probe/exercise distinction (Section 5.3) is the software translation. Verdict: the conceptual model exists, in another kingdom of engineering entirely.

Backup and disaster-recovery practice. The one software-adjacent field with genuine dormancy muscle: restore testing, DR drills, immutable snapshots, air-gapped copies. DR is the *founding* use case of this paper's D2 class, and the better enterprises run annual restore tests. But DR tooling is scoped to *data* recovery within vendor ecosystems — its wake probes are restore jobs, its contracts are RTO/RPO — and nothing generalizes to the boot stick's TLS handshake, the radio firmware's endpoint, the evidence

Table 1: Table 1: the landscape, graded.

System	Models time	Exercise	Credential clock	Media plan	Knowledge plan	Wake bound contract
SRE continuous verification	running time	continuous	rotation built-in	n/a	runbooks + game days	SLO (while running)
Software Heritage / OAIS	archival time	none (preservation)	out of scope	mature	designated community	access, unbounded
MIL storage standards	shelf time	calendar inspection	partial	preservation packaging	institutional	implied
Seed-bank germination	shelf time	viability tests	n/a	cold storage, multi-copy	institutional	implied
Backup/DR drills	recovery time	restore tests	partial	immutable snapshots	DR runbooks	RTO/RPO for <i>data</i>
Container snapshots	none (implicit)	none	image CVE churn	registry deletion risk	none	none
This spec	dormancy time	probe/exercise ladder	ledger + wake probe	multi-media, check- summed	succession pointer	declared wake bound

tool’s format readers, or the knowledge channel. Verdict: the strongest existing fragment; domain-bound to backup/restore.

Containerization and its illusion. The reflexive modern answer: “containerize it; images are portable.” The illusion dissolves on inspection — registries delete unused images (the major registry’s free-tier purge events are public; details verification-queued), base images churn under CVE pressure, the image format itself moves, and the runtime that boots the image assumes a kernel that drifts. A container is a *snapshot of an environment contract*, and the contract’s other party keeps editing the terms. Verdict: a dormancy *mechanism* that pretends to be a dormancy *guarantee*.

Components PARTIAL-to-STRONG on both flanks; the middle regime vacant. The falsifiable claim: *no deployed system declares a dormancy class, maintains a drift ledger, schedules escalating probes, and contracts a wake bound for a non-running software artifact*. One counterexample refutes it.

4 Why the gap survived

The always-on regime won the culture. The industry’s center of gravity moved decisively toward continuously-exercised systems: cloud-native, microservices, observability, chaos engineering — all fluent in the running state and structurally mute about the stopped one. The stopped state became a *failure* to be paged about, not a *regime* to be engineered — so the artifacts that are supposed to be stopped (the emergency kit, the break-glass path) inherited no vocabulary. Their owners file them under “backup,” a word whose practice covers data and dissolves before software.

Dormancy failures are invisible until they are catastrophic. An always-on system’s failure pages someone at 3 a.m. and gets fixed; a dormant system’s failure is discovered *inside the emergency it was built for*, when the marginal cost of discovery is the event itself. The feedback loop is inverted: the worse the rot, the quieter the years that hide it. No telemetry, no page, no ticket, no burn-down — the organizational absence this program’s other papers keep cataloguing (defaults were ungoverned because invisible; complexity was unaccounted because diffuse; here the root is *temporal*: the failure mode only manifests when nothing else is working).

Certification chose snapshots over processes. Compliance regimes audit *the state of artifacts* — the sealed kit exists, the runbook is filed, the evidence vault is locked — because states are photographable.

Dormancy assurance is a *process property*: a schedule kept, a ledger maintained, a probe history proving the artifact stayed alive. A framework that photographs cannot certify a heartbeat, so the market never demanded one.

The economics looked trivial and weren't. “Just boot it once a year” is the folk answer, and it is correct as far as it goes — the way “just eat less” is the folk answer to a metabolic epidemic. The unpriced parts: who owns the calendar across staff changes; what “boot” means when the exercise needs an isolated network; what counts as *pass* (it started? it started and reached its first dependency? it completed a representative workload?); what happens on *partial* pass; and who renews the credentials the clock keeps killing. The folk answer outsources all of that to heroism, and heroism does not survive reorganizations.

The container illusion arrived at the perfect time. Right as practitioners might have noticed the drawer, the industry handed them a story in which the problem no longer existed. The story is load-bearing and false (Section 3), and it will take a decade of registry-purge and endpoint-churn casualties to falsify it publicly at scale.

5 The specification

5.1 The readiness manifest

Every dormant artifact ships with a machine-readable manifest — the wake contract's terms:

```
dormancy: { class: D2, wake_bound: 4h, exercise: semiannual, probes: [monthly],
owner: role-not-person, succession: pointer, environment: { min_kernel, endpoints[],
crypto[], formats[] }, credentials: { inventory, expiry_ledger }, media: {
copies, media_types, checksums } }
```

Three design decisions inside the fields. *Owner is a role, not a person* — the knowledge-decay channel's first countermeasure, borrowed from the bus-factor protocol's legibility rules (P-010). *Environment is enumerated as a capability list* — the drift ledger's seed. *Credentials are inventoried with expiries* — the clock's channel made visible before it bites. The manifest is itself versioned and checksummed in every media copy; a manifest that cannot be found or trusted is a wake failure discovered at wake time, which is one wake failure too late.

5.2 The environment-drift ledger

The artifact's environment section is maintained as a *ledger of observed world-changes*: endpoint monitor results, deprecation notices from dependencies, TLS capability surveys. The wake probe ladder (5.3) updates it; between probes, the ledger's *model* extrapolates: endpoints with historical annual removal probability p are risk-scored as the dormancy interval grows. This is the actuarial heart of the discipline — $W(D, T)$ is computable from the ledger without waking the artifact, and *that* is what turns the readiness contract from a promise into a forecast. The ledger's data sources are deliberately boring (availability probes, vendor deprecation feeds, certificate transparency logs for the crypto section), and its aggregation is deliberately cheap: a dormant artifact's monitoring budget should be a rounding error against a running service's, or the discipline prices itself out of its own market.

5.3 The wake-probe ladder

Probing at three depths, escalating:

- **P1 — presence probes** (cheap, monthly): checksum media copies; verify manifest integrity; confirm key custody. No execution. Catches media decay and custody loss.
- **P2 — capability probes** (quarterly): from an *isolated* environment, exercise the artifact's external dependencies — TLS handshakes against its endpoints, API shape checks, DNS reachability, root-store validation of its certificate chains. No full boot of the artifact. Catches capability drift and credential expiry — the two clock channels — at their *source*, before the artifact's own state matters.
- **P3 — exercise** (semiannual or annual): full wake in an isolated environment, against mocked or sandboxed versions of the live world, completing a *representative workload*: the boot stick restores a canary image; the enrollment kit enrolls a synthetic student; the evidence tool decrypts and produces a test container. Pass criteria declared in advance in the manifest — it started; it reached first dependency; it completed workload within wake bound $\times 2$ (the doubling is honest: an isolated exercise is a rehearsal, not the performance).

The P2/P3 distinction is the seed bank's germination insight generalized: *viability* is measurable without *planting the whole field*. A full exercise is expensive and mildly risky (secrets exposed to a sandbox, calendars consumed); the ladder buys most of the assurance at a fraction of the cost, and the calendar (5.4) spends the full exercise only when probe evidence says the world moved.

5.4 The exercise calendar and escalation policy

The calendar is the discipline's spine: P1 monthly, P2 quarterly, P3 semiannual, with *event-driven escalation* — a ledger entry showing endpoint churn, a root expiry, a dependency deprecation triggers the next-depth probe immediately. The policy is an event-sourced audit trail: every probe appended with actor, timestamp, result; missed probes accumulate as *burned readiness* visible in the next audit (the error-budget's dormancy twin). The succession pointer in the manifest designates the probe's runner as a role; calendar systems already handle role-based assignment, and the manifest's succession field links to the bus-factor machinery's succession registry (P-010) when one exists upstream.

5.5 The readiness contract and the audit form

The contract binds three parties: the owner declares class, wake bound, and cadence; the auditor (internal or insurer-grade) verifies the probe history against the manifest; and the *consumer* of the artifact — the hospital administrator, the court, the emergency network's coordinator — receives the only artifact that matters: a dated, signed statement of the form “*This system is warranted to wake within 4 hours and complete its declared workload, with wake-time assurance estimated at 0.97, last exercised 2026-04-11, drift ledger current through 2026-10.*” The audit form is deliberately short — history table, ledger currency, credential state, knowledge-channel check (succession pointer answered within 72h in a challenge test) — because audit forms that are long become shelf documents, and shelf documents are this discipline's enemy. Procurement language is the adoption wedge: any vendor selling “cold standby,” “break-glass,” or “offline recovery” can be held to the contract's terms, which is precisely the language no current vendor can sign — the gap's market shape, made executable.

5.6 Design for dormancy: the artifact-side rules

The protocol above meets the artifact halfway; artifacts can be *built* to sleep better, and the rules are cheap at design time and ruinous to retrofit:

- **Clock independence:** never let correctness depend on wall-clock trust at wake; handle expired-cert failure with an explicit degraded path rather than a crash (the wake probe will find it either way — the rule makes the finding survivable).
- **Endpoint pinning with declared alternates:** ship the capability list *and* fallbacks; the ledger tracks both.
- **Bundled secrets with expiry-laddering:** rotate *before* dormancy into the longest-lived credential class the counterparty supports, and record the ladder in the manifest.
- **Media diversity:** never a single USB; the multi-copy rule with at least two media families (optical + flash, or flash + paper-encoded for the smallest critical secrets).
- **The manifest is the artifact:** every copy ships the manifest; every exercise updates it; a found-without-manifest artifact is *presumed dead* until proven otherwise — the presumption that makes honest discovery the default.

6 Evaluation design

1. **Dormancy audit of real kits.** Recruit 20 real D2 artifacts (break-glass runbooks, sealed emergency media, legal-hold tooling, radio firmware images) across organizations; run the P2/P3 ladder against them; report pass rates by decay channel. Hypothesis: majority failure, dominated by capability drift and credential expiry — the audit *itself* is the paper’s headline instrument, since the current pass rate is unknown precisely because nobody measures it.
2. **Ledger forecast validation.** For artifacts with probe history, test whether W(D, T) forecasts from the drift ledger predict exercise outcomes (calibration curves, Brier scores on pass/fail). Success: forecast beats a base-rate guess.
3. **Calendar adherence study.** Deploy the manifest+calendar pattern with three organizations for 18 months; measure probe adherence, escalation latency, and the succession challenge’s response rate.
4. **Wake economics.** Cost accounting across the ladder: P1/P2/P3 costs versus the always-on alternative and the no-maintenance baseline, at D2 and D3. Success: the ladder dominates both at realistic event probabilities.

7 What this is not

Not high availability. HA keeps systems running through failure; dormancy engineering keeps *stopped* systems wakeable through time. The tools overlap (probes, drills) and the contracts differ (uptime SLO vs wake bound). **Not archival.** Preservation guarantees the bits exist; this discipline guarantees the bits *work* — OAIS composition is welcome, and its criteria (designated community, ingest discipline) are borrowed, but the success predicate differs. **Not a backup product.** DR tooling covers data recovery well; the specification generalizes to artifacts whose wake depends on live environments, credentials, and humans — the class DR does not model. **Not a container-scanning vendor.** Image CVE scanners audit *contents*; the drift ledger audits the *world’s side of the contract* — the registry’s and endpoint’s behavior over calendar time, which no current scanner models. **Not rotation-only credential hygiene.** Rotation keeps running systems fresh; the ladder keeps sleeping systems’ credentials *valid* or their expiry *known*, which is a different actuarial object.

8 Objections, confronted

“Just keep it running.” The counsel of the always-on regime, and it fails three ways: the artifact’s value often *is* its disconnection (the air gap is the security model); running costs continuous money against a conditional event (the economics invert exactly at D2); and the always-on copy is not the dormant artifact — the sealed kit, the escrowed image, and the evidence container have identity and legal meaning that a parallel running instance does not share. **“Containers solve this.”** Section 3’s analysis: registries purge, bases churn, kernels drift — the image freezes the artifact’s side of a contract whose other party keeps editing. The drift ledger is precisely the missing other side. **“Test it once a year — how hard?”** Section 4 priced the folk answer: ownership across reorganizations, isolation for the exercise, pass criteria, partial passes, credential renewal — the calendar is the difference between an intention and an institution. **“Nobody will pay for dormant assurance.”** They already pay for its absence — inside the emergencies the kits exist for — but the bill arrives unpriced and unlabeled, which is the feedback-visibility problem of Section 4. The readiness contract’s procurement wedge converts the unpriced bill into a line item vendors compete on. **“Air-gapped artifacts can’t have ledgers/probes.”** The probes run from *adjacent* semi-isolated stations against mirrored world-state; the ledger lives outside the gap and predicts across it. The air gap constrains *data flow in*, not *assurance flow around*.

9 Limitations

The environment model is an extrapolation over a partially observable world — endpoints die without notice, and the ledger’s forecasts carry the honest uncertainty of any actuarial table (the evaluation’s Brier-score instrument exists because the forecasts will sometimes be wrong). Exercise fidelity is bounded: a sandboxed P3 workload is a rehearsal, and real wakes will find conditions the rehearsal mocked away — the wake-bound doubling in 5.3 is the acknowledgment, not a fix. The knowledge channel resists full mechanization: the succession challenge checks that *someone* answers, not that the answer is right; deeper verification costs exactly the human attention whose decay is being measured. The D3 regime inherits cryptographic epoch risk (the migration of the world’s roots and algorithms) that no ladder shorter than re-instrumentation fully covers. And the discipline’s adoption depends on the audit ecosystem — without insurers or procurement demanding the contract, the manifest is a document drawer-bound next to the artifact it describes; the DEPA-style lesson from P-017 applies: regulated perimeters adopt protocols first, and the general web follows only when the priced silence gets loud. Citations from domain knowledge carry *verification-queued* status through the live-source ledger.

10 Conclusion

The drawer is not neutral. Everything in it — the boot stick, the sealed firmware, the evidence tool, the runbook’s six laminated steps — is running a race against a world that edits itself on a clock, credentials that die politely in the dark, media that forgets, and people who move on. The industry’s two mature regimes, always-on and archival, cover the race’s two banks and none of its middle: the artifacts whose entire purpose is to sleep through the years and wake in the emergency. Dormancy engineering gives that middle a class system, an actuarial ledger for the world’s churn, a probe ladder that measures viability without planting the field, a calendar that survives reorganizations, and a contract that lets a hospital administrator, a judge, or an emergency coordinator read one sentence — *this will wake within four hours; here is the proof it still can* — and act on it. The seed vaults already run this discipline for germplasm; the arsenals ran it for crates; the DR rooms almost ran it for data. Software — the most churn-dependent artifact class humanity has ever produced — is the last one still filing its emergency kits under a laminated promise. The promise is now a protocol.

References

Verification-queued marks follow the program’s citation-honesty convention.

1. Google SRE Team. *Site Reliability Engineering* (error budgets, game days, chaos practice). O’Reilly, 2016.
2. ISO. *ISO 14721: Open Archival Information System (OAIS)* — reference model.
3. Software Heritage. *The Software Heritage Archive* — content-addressed source preservation. swheritage.org.
4. Abrams, S., et al. on emulation-based re-execution of preserved software. (EaaS program publications; verification queued.)
5. United States Department of Defense. *MIL-STD storage and preservation packaging standards* (calendar-inspection tradition; verification queued.)
6. Svalbard Global Seed Vault / NordGen. *Germination testing procedures for stored seed lots*. (verification queued)
7. ISRG / Let’s Encrypt. *DST Root CA X3 expiry and legacy-client breakage, September 2021*. (verified 2026-10: Let’s Encrypt’s own expiration notice — root expired September 30, 2021; older devices trusting only DST Root CA X3 broke.)
8. Docker Inc. inactive-image deletion policy for free tiers — announced 2020 (6-month inactivity rule), with terms of service and retention tightened 2023–24. (verified 2026-10 as a policy class; date range corrected — the original announcement is 2020, the tightening 2023–24.)
9. Snedaker, S. on business continuity and DR testing cadence. (verification queued)
10. Vovk, P. et al. on cold-storage cryptocurrency custody procedures. (verification queued)
11. Granger, S. “Emulation as a Digital Preservation Strategy.” *D-Lib Magazine*, 6(10), October 2000; and Rosenthal, D. S. H. “Emulation & Virtualization as Preservation Strategies” (LOCKSS Program). (verified 2026-10; attribution corrected — the D-Lib title is Granger’s, not Rosenthal’s, and dates from 2000, not 2015.)
12. Gray, J., van Ingen, C. *Empirical Measurements of Disk Failure Rates and Error Rates*. MSR-TR-2005-166. (media-failure priors; verified 2026-10: the MSR technical report and its arXiv mirror.)
13. p-rick research program. *P-008 The Afterlife of Devices; P-010 The Bus Factor Protocol; P-011 Model Extinction; P-004 Degradation Contracts*. 2026.
14. RFC 8446 / TLS capability evolution literature. (handshake-version drift priors; verification queued)
15. Jones, C. *Applied Software Measurement* (maintenance-to-development cost ratios; verification queued)
16. Backblaze. *Drive-stats and flash-decay annual reports*. (media priors; verified 2026-10: the Drive Stats report series is published openly.)